实验题目：实验（二）：Java 基本语法

实验目的

1. 掌握 Java 的基本语法
2. 掌握 Java 的流程控制
3. 掌握数组的使用

实验要求

1. 每个题目，写出实验思路
2. 代码实现
3. 运行结果截图

实验内容

1. 编写一个 Java 应用程序，给出汉字'你'、'我'、'他'在 Unicode 表中的位置。
2. 编写一个 Java 应用程序，输出全部的希腊字母。
3. 编写一个 Java 应用程序，计算$1!+2!+3!+...+10!$。
4. 编写一个 Java 应用程序，计算 1-50 之间的所有素数的和。
5. 分别用 do-while 和 for 循环计算$1+\frac{1}{2!}+\frac{1}{3!}+\frac{1}{4!}$..... 的前 20 项和。
6. 一个数如果恰好等于它的因子之和，这个数就称为"完数"。例如 6=1＋2＋3。编程求 1000 之内的所有完数。
7. 编写程序，求出几何形状（长方形、正方形、圆形）的周长和面积。
8. 编写一个 Java 应用程序，产生 10 个 100 之内的随机整数输出，并把这 10 个数从小到大的顺序输出。
9. 编写一个 Java 应用程序，产生 20 个 50-100 之内的整数，并输出这 20 个数并找出最大数及最小数输出。
10. 输入一行字符，分别统计出其中英文字母、空格、数字和其它字符的个数。（可以使用 Scanner 对象的 nextLine 方法获取一个字符串，然后调用字符串的 toCharArray 方法将字符串转化为字符数组进行处理）

注：题目中可能需要用到有些类和方法，大家还没有学过，请查阅相关资料，作为自学部分，比如随机输出整数等。

实验执行

任务一：汉字在 Unicode 表中的位置

（1）思路

可以利用 Java 的字符处理特性获取汉字的 Unicode 编码值。将汉字字符转换为整数可得到十进制值，再使用 Stream 和方法引用进行转换可以获得十六进制表示。

（2）代码实现

```
import java.util.stream.Stream;

public class UnicodeLocator {
    public static void main(String[] args) {
```

```
        // 使用函数式编程处理字符的Unicode值
        Stream.of('你', '我', '他')
            .forEach(character -> {
                String hexValue = Integer.toHexString((int) character);
                System.out.printf("汉字'%c'的Unicode位置: %d (十六进制: %s)%n",
                                  character, (int) character, hexValue);
            });
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
UnicodeLocator.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
UnicodeLocator
汉字'你'的 Unicode 位置: 20320 (十六进制: 4f60)
汉字'我'的 Unicode 位置: 25105 (十六进制: 6211)
汉字'他'的 Unicode 位置: 20182 (十六进制: 4ed6)
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

任务二：输出全部的希腊字母

（1）思路

希腊字母在 Unicode 中占据特定范围。我们可以用函数式编程方式创建这个范围的流，然后将每个码点转换为字符并输出。大写希腊字母从 U+0391 到 U+03A9，小写从 U+03B1 到 U+03C9。

（2）代码实现

```
import java.util.function.IntConsumer;
import java.util.stream.IntStream;

public class GreekLetterDisplay {
    public static void main(String[] args) {
        // 函数式处理输出逻辑
        IntConsumer printChar = codePoint -> System.out.print((char) codePoint + "
");

        System.out.println("大写希腊字母:");
        IntStream.rangeClosed(0x0391, 0x03A9)
                .forEach(printChar);

        System.out.println("\n小写希腊字母:");
        IntStream.rangeClosed(0x03B1, 0x03C9)
                .forEach(printChar);
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
GreekLetterDisplay.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
GreekLetterDisplay
```

```
大写希腊字母：
Α Β Γ Δ Ε Ζ Η Θ Ι Κ Λ Μ Ν Ξ Ο Π Ρ Σ Τ Υ Φ Χ Ψ Ω
小写希腊字母：
α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ ς σ τ υ φ χ ψ ω
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

任务三：计算 1!+2!+3!+...+10!

（1）思路

使用函数式编程计算阶乘之和。先编写一个计算阶乘的纯函数，然后创建一个从 1 到 10 的流，对每个数计算其阶乘并求和。

（2）代码实现

```
import java.util.stream.LongStream;
import java.util.function.LongUnaryOperator;

public class FactorialSummation {
    public static void main(String[] args) {
        // 阶乘计算函数
        LongUnaryOperator calculateFactorial = n ->
            LongStream.rangeClosed(1, n).reduce(1, (a, b) -> a * b);

        // 计算并展示每个阶乘
        LongStream.rangeClosed(1, 10)
            .peek(n -> System.out.printf("%d! = %d%n", n,
calculateFactorial.applyAsLong(n)))
            .map(calculateFactorial)
            .sum();

        // 计算总和
        long result = LongStream.rangeClosed(1, 10)
            .map(calculateFactorial)
            .sum();

        System.out.printf("1! + 2! + 3! + ... + 10! = %d%n", result);
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
FactorialSummation.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
FactorialSummation
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
```

```
9! = 362880
10! = 3628800
1! + 2! + 3! + … + 10! = 4037913
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

任务四：计算 1-50 之间的所有素数的和

（1）思路

利用函数式编程创建一个判断素数的谓词函数，然后使用流处理从 1 到 50 的数，筛选出素数并求和。

（2）代码实现

```
import java.util.function.IntPredicate;
import java.util.stream.IntStream;

public class PrimeSummation {
    public static void main(String[] args) {
        // 创建判断素数的谓词函数
        IntPredicate isPrimeNumber = num -> {
            if (num <= 1) return false;
            return IntStream.rangeClosed(2, (int) Math.sqrt(num))
                .noneMatch(divisor -> num % divisor == 0);
        };

        // 筛选素数并收集
        int[] primes = IntStream.rangeClosed(2, 50)
            .filter(isPrimeNumber)
            .toArray();

        // 输出素数
        System.out.print("1-50之间的素数: ");
        IntStream.of(primes).forEach(prime -> System.out.print(prime + " "));

        // 计算和输出和
        int sum = IntStream.of(primes).sum();
        System.out.println("\n1-50之间素数之和: " + sum);
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
PrimeSummation.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
PrimeSummation
1-50 之间的素数: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
1-50 之间素数之和: 328
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

任务五：计算级数和（$1+\frac{1}{2!}+\frac{1}{3!}+\ldots$）的前 20 项和

（1）思路

使用函数式编程计算阶乘倒数之和。分别用函数式的 for 和 do-while 两种风格实现，比较结果。

（2）代码实现

```
import java.util.function.DoubleSupplier;
import java.util.stream.IntStream;

public class SeriesSummationCalc {
    public static void main(String[] args) {
        // 函数式风格使用Stream API计算
        double streamSum = IntStream.rangeClosed(1, 20)
            .mapToDouble(i -> {
                double denominator = IntStream.rangeClosed(1, i-1)
                    .asDoubleStream()
                    .reduce(1.0, (a, b) -> a * b);
                return 1.0 / (i == 1 ? 1.0 : denominator);
            })
            .sum();

        // 模拟命令式do-while的函数式实现
        DoubleSupplier doWhileStyle = () -> {
            double result = 0.0;
            double factorialValue = 1.0;
            int index = 1;

            do {
                if (index > 1) {
                    factorialValue *= (index - 1);
                }
                result += 1.0 / factorialValue;
                index++;
            } while (index <= 20);

            return result;
        };

        System.out.println("使用Stream计算得到的前20项和： " + streamSum);
        System.out.println("使用函数式do-while计算得到的前20项和： " +
doWhileStyle.getAsDouble());
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
SeriesSummationCalc.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
SeriesSummationCalc
使用 Stream 计算得到的前 20 项和： 2.7182818284590455
使用函数式 do-while 计算得到的前 20 项和： 2.7182818284590455
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

任务六：求 1000 之内的所有完数

（1）思路

使用函数式编程查找完数。定义一个函数获取数的所有因子，然后判断因子之和是否等于数本身。使用流处理遍历 1 到 1000 的数字。

（2）代码实现

```
import java.util.List;
import java.util.function.IntFunction;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

public class PerfectNumberFinder {
    public static void main(String[] args) {
        System.out.println("1000以内的完数:");

        // 查找因子的函数
        IntFunction<List<Integer>> getDivisors = n ->
            IntStream.range(1, n)
                .filter(i -> n % i == 0)
                .boxed()
                .collect(Collectors.toList());

        // 找出所有完数
        IntStream.rangeClosed(1, 1000)
            .filter(n -> {
                List<Integer> divisors = getDivisors.apply(n);
                return divisors.stream().mapToInt(Integer::intValue).sum() == n;
            })
            .forEach(perfectNum -> {
                List<Integer> factors = getDivisors.apply(perfectNum);
                System.out.print(perfectNum + " = " + factors.get(0));
                factors.stream().skip(1).forEach(f -> System.out.print(" + " + f));
                System.out.println();
            });
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
PerfectNumberFinder.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
PerfectNumberFinder
1000以内的完数:
6 = 1 + 2 + 3
28 = 1 + 2 + 4 + 7 + 14
496 = 1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

任务七：几何形状的周长和面积

（1）思路

使用函数式接口和 Lambda 表达式定义几何图形的计算方法，采用接口默认方法和静态方法的特性，构建一个函数式风格的形状计算系统。

（2）代码实现

```java
import java.util.function.BiFunction;
import java.util.function.Function;

public class GeometryCalculator {
    public static void main(String[] args) {
        // 使用函数式接口表示形状
        BiFunction<Double, Double, Shape> createRectangle = (l, w) ->
            new Shape(
                () -> 2 * (l + w),
                () -> l * w,
                "长方形 (长=" + l + ", 宽=" + w + ")"
            );

        Function<Double, Shape> createSquare = side ->
            new Shape(
                () -> 4 * side,
                () -> side * side,
                "正方形 (边长=" + side + ")"
            );

        Function<Double, Shape> createCircle = radius ->
            new Shape(
                () -> 2 * Math.PI * radius,
                () -> Math.PI * radius * radius,
                "圆形 (半径=" + radius + ")"
            );

        // 创建形状实例并计算
        Shape rect = createRectangle.apply(5.0, 4.0);
        Shape square = createSquare.apply(5.0);
        Shape circle = createCircle.apply(3.0);

        // 显示计算结果
        System.out.println(rect.name);
        System.out.println("周长: " + rect.getPerimeter());
        System.out.println("面积: " + rect.getArea());

        System.out.println("\n" + square.name);
        System.out.println("周长: " + square.getPerimeter());
        System.out.println("面积: " + square.getArea());

        System.out.println("\n" + circle.name);
        System.out.println("周长: " + circle.getPerimeter());
        System.out.println("面积: " + circle.getArea());
    }

    // 形状类使用函数式接口
    static class Shape {
        private final PerimeterCalculator perimeterCalc;
        private final AreaCalculator areaCalc;
        final String name;

        Shape(PerimeterCalculator p, AreaCalculator a, String name) {
            this.perimeterCalc = p;
            this.areaCalc = a;
```

```
            this.name = name;
        }

        double getPerimeter() {
            return perimeterCalc.calculate();
        }

        double getArea() {
            return areaCalc.calculate();
        }
    }

    @FunctionalInterface
    interface PerimeterCalculator {
        double calculate();
    }

    @FunctionalInterface
    interface AreaCalculator {
        double calculate();
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
GeometryCalculator.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
GeometryCalculator
长方形 (长=5.0, 宽=4.0)
周长: 18.0 面积: 20.0

正方形 (边长=5.0)
周长: 20.0 面积: 25.0

圆形 (半径=3.0)
周长: 18.84955592153876 面积: 28.274333882308138
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

任务八：产生 10 个 100 之内的随机整数并排序

（1）思路

使用 Java 8 的流 API 生成随机数，并利用函数式编程对它们进行排序和处理。

（2）代码实现

```
import java.util.Arrays;
import java.util.Random;
import java.util.stream.IntStream;

public class RandomNumberProcessor {
    public static void main(String[] args) {
        // 创建随机数生成器
        Random randomGenerator = new Random();

        // 生成10个随机数并收集到数组
```

```
        int[] randomValues = IntStream
            .generate(() -> randomGenerator.nextInt(100))
            .limit(10)
            .toArray();

        // 显示原始随机数
        System.out.println("生成的随机数序列:");
        IntStream.of(randomValues)
            .forEach(num -> System.out.print(num + " "));

        // 排序处理
        int[] sortedValues = Arrays.stream(randomValues)
            .sorted()
            .toArray();

        // 显示排序结果
        System.out.println("\n排序后的序列:");
        IntStream.of(sortedValues)
            .forEach(num -> System.out.print(num + " "));
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
RandomNumberProcessor.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
RandomNumberProcessor
生成的随机数序列:
23 58 17 91 45 8 71 36 64 29
排序后的序列:
8 17 23 29 36 45 58 64 71 91
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

任务九：产生 20 个 50-100 之内的整数并找出最大值和最小值

（1）思路

使用函数式编程生成指定范围内的随机数，并利用流处理 API 找出最大值和最小值。

（2）代码实现

```
import java.util.IntSummaryStatistics;
import java.util.Random;
import java.util.stream.IntStream;

public class RandomRangeAnalyzer {
    public static void main(String[] args) {
        Random randomSource = new Random();

        // 使用映射生成50-100范围的随机数
        int[] randomNumbers = IntStream
            .generate(() -> randomSource.nextInt(51) + 50)
            .limit(20)
            .toArray();
```

```
        // 显示生成的随机数
        System.out.println("生成的20个随机数（50-100范围）:");
        IntStream.of(randomNumbers)
            .forEach(num -> System.out.print(num + " "));

        // 使用统计功能找出最大值和最小值
        IntSummaryStatistics stats = IntStream.of(randomNumbers)
            .summaryStatistics();

        System.out.println("\n数列最大值: " + stats.getMax());
        System.out.println("数列最小值: " + stats.getMin());
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
RandomRangeAnalyzer.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
RandomRangeAnalyzer
生成的 20 个随机数（50-100 范围）:
73 86 55 97 61 78 93 69 84 52 71 89 63 76 58 95 82 67 91 59
数列最大值: 97 数列最小值: 52
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

任务十：统计字符种类

（1）思路

使用函数式编程风格和流 API 处理输入字符串，统计不同类型字符的数量。

（2）代码实现

```
import java.util.Map;
import java.util.Scanner;
import java.util.function.Function;
import java.util.stream.Collectors;

public class CharacterAnalyzer {
    // 字符分类枚举
    enum CharType { LETTER, DIGIT, SPACE, OTHER }

    public static void main(String[] args) {
        Scanner inputReader = new Scanner(System.in);
        System.out.println("请输入一行字符:");
        String text = inputReader.nextLine();

        // 分类函数
        Function<Character, CharType> classifier = c -> {
            if (Character.isLetter(c)) return CharType.LETTER;
            if (Character.isDigit(c)) return CharType.DIGIT;
            if (Character.isWhitespace(c)) return CharType.SPACE;
            return CharType.OTHER;
        };

        // 使用流处理字符分类计数
```

```
        Map<CharType, Long> charCounts = text.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(classifier, Collectors.counting()));

        // 输出结果
        System.out.println("英文字母数量: " + charCounts.getOrDefault(CharType.LETTER,
0L));
        System.out.println("数字数量: " + charCounts.getOrDefault(CharType.DIGIT,
0L));
        System.out.println("空格数量: " + charCounts.getOrDefault(CharType.SPACE,
0L));
        System.out.println("其他字符数量: " + charCounts.getOrDefault(CharType.OTHER,
0L));

        inputReader.close();
    }
}
```

（3）运行结果截图

```
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % javac
CharacterAnalyzer.java
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects % java
CharacterAnalyzer
请输入一行字符：
Hello Java 2023! 函数式编程很酷
英文字母数量: 14
数字数量: 4
空格数量: 4
其他字符数量: 8
(base) houxianzheng@MacBookPro ~/xianzhengJavaProjects %
```

实验测试中遇到的问题和自己的解决方案

1. 函数式编程范式适应：初次接触函数式编程范式时，需要转变思维方式。通过查阅 Java 8 Stream API 文档，逐步适应了函数式编程的思考方式。
2. **Lambda** 表达式性能考虑：在处理大数据量时，发现 Lambda 表达式可能引入一些性能开销。通过适当地选择并行流和优化 Lambda 表达式结构来提高性能。
3. 随机数生成器的使用：使用 Stream API 生成随机数时，需要注意生成器的状态和重用问题。通过创建单一 Random 实例并在流中共享来解决。
4. 浮点数精度问题：在计算阶乘倒数和等数学问题时，发现浮点数精度是一个常见问题。通过使用 BigDecimal 类型可以在需要高精度时解决这个问题。